



**UNIVERSIDAD INTERNACIONAL DEL ECUADOR – UIDE QUITO**  
**CARRERA DE INGENIERÍA EN SISTEMAS DE LA INFORMACIÓN**

**PROYECTO INTEGRADOR:**  
**MINDOLOGIC: DESARROLLO INTERACTIVO**  
**USANDO OPERADORES, CONDICIONALES,**  
**ITERACIONES Y COLECCIONES**

**ESTUDIANTE:**  
**NICOLE ANGELINA SANI GIORDANO**

**MATERIA:**  
**LÓGICA DE PROGRAMACIÓN**  
**FECHA DE ENTREGA: 28/06/2026**

# Índice del documento

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| I.    | INTRODUCCIÓN .....                                      | 1  |
| II.   | DESCRIPCIÓN DEL PROBLEMA .....                          | 1  |
| III.  | RELACIÓN CON LOS CONTENIDOS DE LA ASIGNATURA.....       | 2  |
| i.    | Módulo 1 (Algoritmia, Tipos de Datos y Operadores)..... | 2  |
| ii.   | Módulo 2 (Estructuras de Control Condicionales) .....   | 2  |
| iii.  | Módulo 3 (Estructuras Iterativas).....                  | 2  |
| iv.   | Módulo 4 (Estructuras de Datos – Colecciones) .....     | 2  |
| IV.   | EXPLICACIÓN DEL SISTEMA DESARROLLADO .....              | 3  |
| i.    | Diagrama de Flujo .....                                 | 3  |
| ii.   | Diagrama de Arquitectura.....                           | 4  |
| V.    | ALCANCE DEL SISTEMA A DESARROLLAR.....                  | 4  |
| i.    | Alcance Técnico y Operativo .....                       | 4  |
| VI.   | PROPÓSITO Y REFLEXIÓN SOBRE EL PROYECTO.....            | 4  |
| VII.  | DESARROLLO DEL SOFTWARE .....                           | 5  |
| i.    | Configuración del Repositorio de GitHub.....            | 5  |
| ii.   | Código .....                                            | 6  |
| iii.  | Estructura Lógica del Programa.....                     | 9  |
| VIII. | EJECUCIONES.....                                        | 10 |
| i.    | Ejecución sin Restricciones .....                       | 10 |
| ii.   | Ejecución con Restricciones.....                        | 15 |
| IX.   | ENLACE DEL REPOSITORIO .....                            | 20 |
| X.    | ENLACE DEL VÍDEO .....                                  | 20 |
| XI.   | CONCLUSIONES.....                                       | 20 |
| XII.  | RECOMENDACIONES .....                                   | 21 |
| XIII. | BIBLIOGRAFÍA.....                                       | 21 |

# **I. INTRODUCCIÓN**

El presente proyecto busca diseñar, desarrollar y documentar la evolución de un programa de software propio desarrollado cómo proyecto final del curso de Lógica de Programación, con el nombre de “Un Rescate en Mindo, este sistema busca transitar de un prototipo inicial basado en una interfaz de línea de comandos hacia una aplicación de escritorio interactiva por medio de una interfaz Gráfica de Usuario.

Se termina de implementar una mejor arquitectura basada en eventos, que permite a los usuarios tomar decisiones lógicas estructuradas para resolver acertijos algorítmicos para lograr así rescatar a los tres personajes del juego (Lucky, Chester y Nala), de manera en que se implementan los fundamentos teóricos de la programación estructurada, cómo las expresiones Booleanas (AND, OR, XOR) y el control de flujo, así materializando un software interactivo, sólido y escalable.

# **II. DESCRIPCIÓN DEL PROBLEMA**

Actualmente las tecnologías emergentes influyen drásticamente en la sociedad actual, determinan la forma en que las personas interactúan, trabajan y resuelven problemas del día a día, este proyecto busca desarrollar una solución informática sencilla que responda a la necesidad sobre la dificultad que poseen personas externas a asimilar el mundo de la lógica computacional para entender conceptos básicos cómo los son las compuertas lógicas, entre otros, todo esto mediante un programa interactivo, dinámico y sencillo.

Este sistema busca transformar estos conceptos un poco abstractos, en una experiencia gráfica, intuitiva y manejable, proveyendo así un entorno interactivo que otorga una retroalimentación a los usuarios, promoviendo así el pensamiento crítico y analítico para la toma de decisiones que son tan importantes para el desenvolvimiento en la era actual digital.

### **III. RELACIÓN CON LOS CONTENIDOS DE LA ASIGNATURA**

El programa desarrollado no es un producto aislado, sino que busca en un solo aplicativo integrar de manera teórica y aplicada, los conceptos fundamentales dictados en los diferentes módulos del curso.

#### **i. Módulo 1 (Algoritmia, Tipos de Datos y Operadores)**

La base del sistema recae en la correcta definición de las variables utilizadas dentro de los diferentes tipos de datos estructurados, enseñados en clases, cómo son los “Strings” , los “Booleanos” además de la aplicación de los diferentes operadores lógicos cómo “And”, “Or” y el operador ^ para la compuerta “XOR”, así como el uso de operadores relacionales cómo “==” para construir las reglas que rigen la lógica del rescate de los 3 personajes.

#### **ii. Módulo 2 (Estructuras de Control Condicionales)**

La arquitectura de decisiones del programa se fundamenta en estructuras selectivas cómo “if, else y elif” estas evalúan las condiciones de victoria o fracaso en cada zona específica.

#### **iii. Módulo 3 (Estructuras Iterativas)**

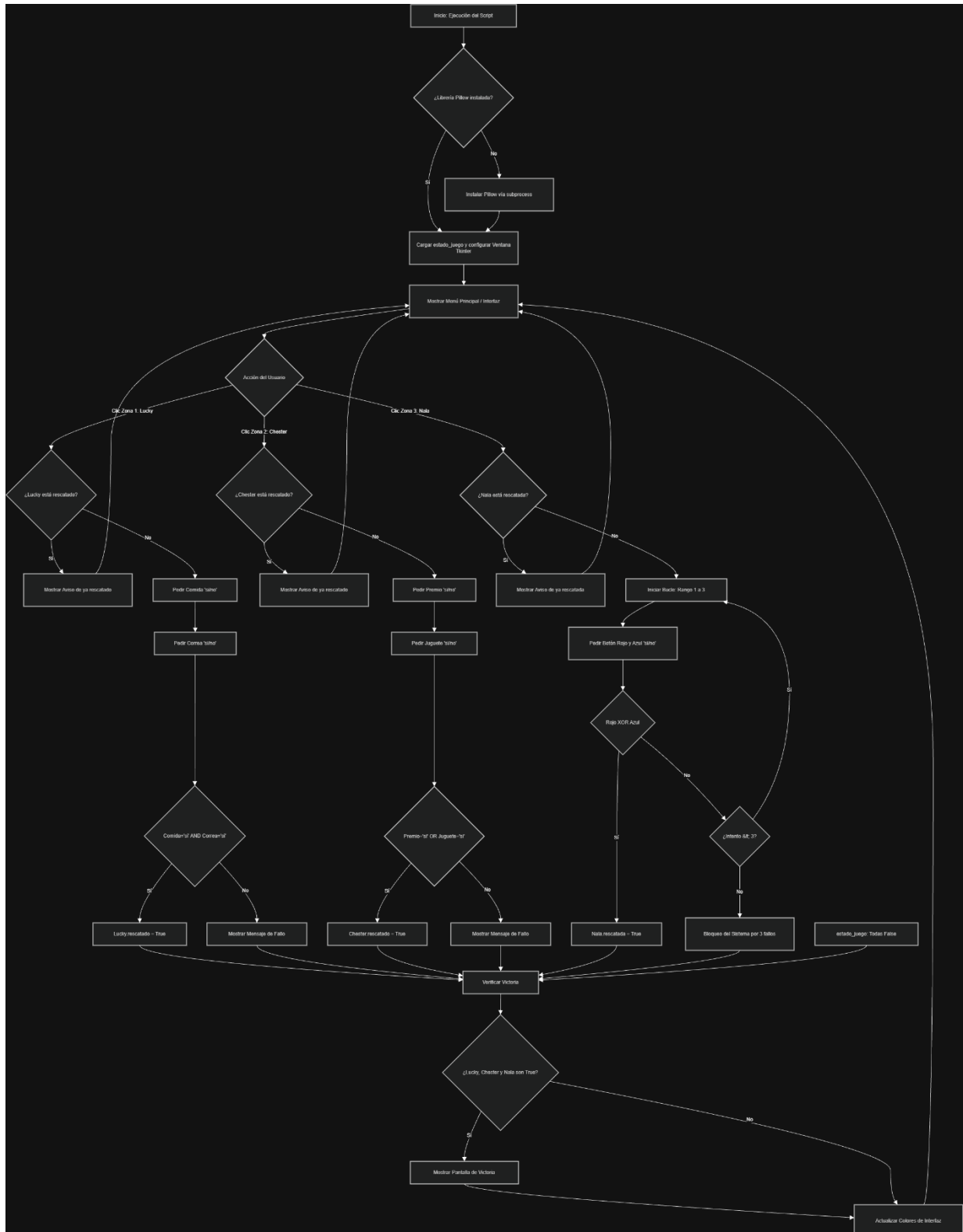
Dentro del programa se implementaron múltiples bucles “while” para garantizar la validación de los datos, aplicando restricciones asegurándonos que el programa no falle en caso de que el usuario ingrese texto no definido dentro del mismo, adicionalmente se aplicó un bucle “for” en la Zona 3 del juego, lo que nos permite limitar los intentos del usuario a un máximo de 3 y detener el ciclo inmediatamente si se pasaba de este límite.

#### **iv. Módulo 4 (Estructuras de Datos – Colecciones)**

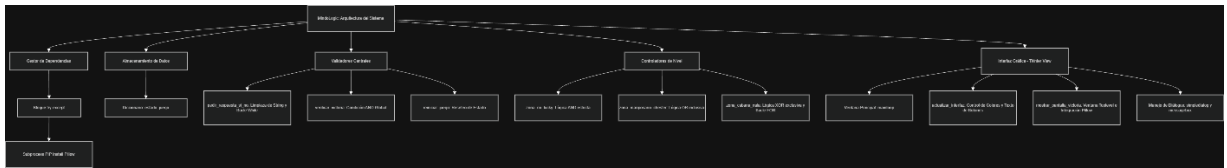
Se sustituyeron las variables booleanas aisladas por un diccionario principal, que nos ayuda a centralizar y organizar el estado del rescate de nuestros personajes (True/False) optimizando así dentro de nuestro software el uso de la memoria.

## IV. EXPLICACIÓN DEL SISTEMA DESARROLLADO

### i. Diagrama de Flujo



## ii. Diagrama de Arquitectura



## V. ALCANCE DEL SISTEMA A DESARROLLAR

Con este programa se busca superar esta barrera que existe para la entrada del razonamiento lógico-matemático, todo mediante la creación de este juego interactivo que demostrará a los usuarios empíricamente cómo es que funcionan las decisiones algorítmicas, implementando de igual manera una interfaz tolerante a fallos y que busca siempre ayudar al usuario a comprender cómo funciona la lógica computacional.

### **i. Alcance Técnico y Operativo**

Este programa se delimita a la creación de una aplicación de escritorio contenida en solo script de ejecución muy sencillo, en este tenemos la interfaz gráfica, la gestión temporal de estados dentro de la memoria RAM, diferentes niveles lógicos independientes, validación de inputs y un sistema de reinicio que limpia la estructura de datos sin terminar la ejecución de nuestro software.

## VI. PROPÓSITO Y REFLEXIÓN SOBRE EL PROYECTO

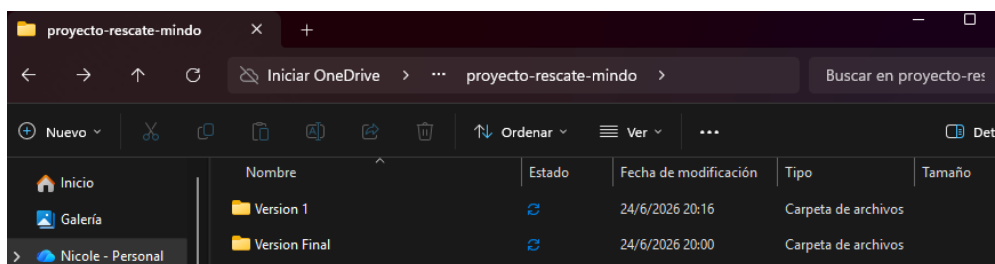
El propósito principal de este software es demostrar que la programación no es simplemente una herramienta para la creación de programas vacíos y planos, sino a su vez, es una herramienta de modelado de la realidad la cuál es capaz no solo de resolver problemas lógicos y operativos sino también, ser un programa educativo que puede ser usado por cualquier usuario y adquirir conocimientos que pueden ser considerados un poco “abstractos” de una manera muy dinámica

Este proyecto refleja cómo el software busca transformar problemas de lógica pura en interfaces interactivas y dinámicas, la integración de sistemas informáticos en la cotidianidad requieren de esto, interfaces que nos ayuden a mediar entre la complejidad de la máquina y la comprensión humana, además es un programa que frente a los errores humanos, no colapsa, sino que siempre busca orientar al usuario mediante instrucciones de retroalimentación, es decir el programa busca empoderar cognitivamente a sus usuarios.

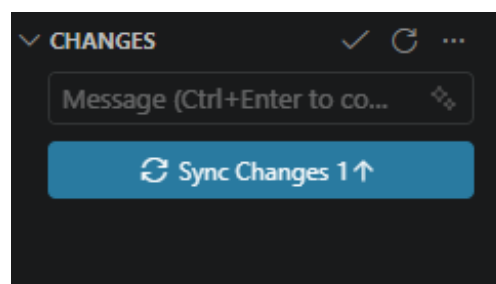
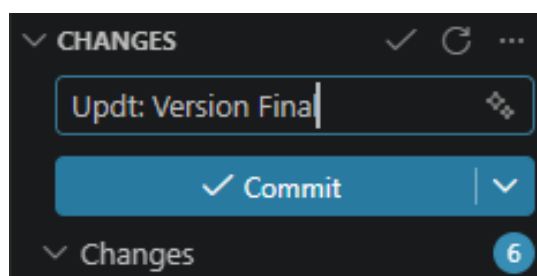
## VII. DESARROLLO DEL SOFTWARE

### i. Configuración del Repositorio de GitHub

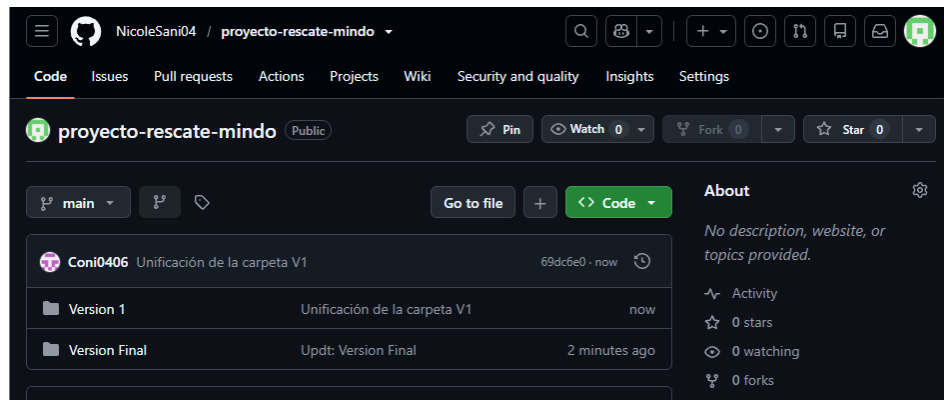
- Para no tener que crear otro repositorio, sino que tener todo el manejo de versiones dentro del mismo repositorio lo que vamos a hacer es, dentro de la carpeta que está conectada al repositorio, crear 2 nuevas carpetas y dividir las en la “Versión 1” (El anterior trabajo realizado) y la “Versión Final” (El trabajo actual).



- Dentro del Visual Studio Code hacemos “Commit” y “Sincronizamos” los cambios realizados que fue la creación de las 2 carpetas para dividir las versiones de nuestro software.



- Abrimos o recargamos nuestro repositorio y se actualizarán automáticamente los cambios que hemos realizado



- Cabe recalcar, que el proyecto actual será trabajado dentro de la carpeta “Versión Final” aquí dentro se encontraran todos los archivos de este trabajo.

## ii. Código

```

projectoFinalLPpy x
Version Final > proyectoFinalLPpy > zona_cabana_nala
1 import sys
2 import subprocess
3 import tkinter as tk
4 from tkinter import messagebox, simpledialog
5
6 # Si la persona que va a jugar no tiene instalada la librería Pillow (para las imágenes),
7 # este bloque la instala automáticamente para que el juego no tenga fallos.
8 try:
9     from PIL import Image, ImageTk
10 except ImportError:
11     subprocess.check_call([sys.executable, "-m", "pip", "install", "pillow"])
12     from PIL import Image, ImageTk
13
14 # Guardamos el progreso del juego en un diccionario.
15 # Así es más ordenado saber qué mascota ya está a salvo.
16 estado_juego = {
17     "Lucky": {"rescatado": False},
18     "Chester": {"rescatado": False},
19     "Nala": {"rescatado": False}
20 }
21
22 # Esta función nos sirve para asegurar que el usuario solo escriba "si" o "no".
23 # Si pone cualquier otra cosa, le lanza un error y le recuerda las instrucciones.
24 def pedir_respuesta_si_no(titulo, pregunta, instrucciones):
25     while True:
26         # Se agrega parent=ventana para que el cuadro de texto NO se vaya para atrás
27         respuesta = simpledialog.askstring(titulo, pregunta, parent=ventana)
28         # Si el usuario cierra la ventana, cancelamos la acción.
29         if respuesta is None:
30             return None
31
32         # Limpiamos el texto: lo pasamos a minúsculas y le quitamos espacios extra.
33         respuesta = respuesta.lower().strip()
34         if respuesta in ["si", "no"]:
35             return respuesta
36         else:
37             # Mostramos el error sin emojis y volvemos a cargar las instrucciones (siempre arriba).
38             messagebox.showwarning("Entrada invalida", "Por favor, responde estrictamente con 'si' o 'no'.", parent=ventana)
39             messagebox.showinfo(f"Recordatorio - {titulo}", instrucciones, parent=ventana)
40
41 # Vuelve a poner a todas las mascotas como no rescatadas (False) para iniciar otra partida.
42 def reiniciar_juego():
43     for mascota in estado_juego:
44         estado_juego[mascota]["rescatado"] = False
45     actualizar_interfaz()
46     messagebox.showinfo("Reinicio Exitoso", "El juego se ha reiniciado, las mascotas estan escondidas de nuevo.", parent=ventana)
47
48 # Esta es la ventana final que aparece cuando ganas el juego, mostrando la imagen editada.

```



```

projectFinalRpy X
Version Final > projectFinalRpy.py > estado_juego

48 # Esta es la ventana final que aparece cuando ganas el juego, mostrando la imagen editada.
49 def mostrar_pantalla_victoria():
50     ventana_victoria = tk.Toplevel(ventana)
51     ventana_victoria.title("Misión Cumplida!")
52     ventana_victoria.geometry("450x620")
53     ventana_victoria.configure(bg="#2f3640")
54
55     # Intentamos cargar la imagen "victoria.jpeg". Si no la encuentra, mostramos un texto de error.
56     try:
57         img_original = Image.open("victoria.jpeg")
58         img_redimensionada = img_original.resize((480, 500), Image.Resampling.LANCZOS)
59         img_tk = ImageTk.PhotoImage(img_redimensionada)
60
61         lbl_img = tk.Label(ventana_victoria, image=img_tk, bg="#2f3640")
62         lbl_img.image = img_tk # Guardamos la imagen en memoria para que no se borre
63         lbl_img.pack(pady=(15, 5))
64     except Exception as e:
65         tk.Label(ventana_victoria, text="Imagen 'victoria.jpeg' no encontrada", fg="#ff7f75", bg="#2f3640").pack(pady=20)
66
67     # Texto de felicitación y botón para cerrar (aquí si conservamos emojis en el diseño).
68     tk.Label(ventana_victoria, text="¡FELICIDADES! ¡Has dominado la lógica básica de la programación!", font=("Segoe UI", 11, "bold"), bg="#2f3640", fg="#fbc531").pack(pady=5)
69     tk.Button(ventana_victoria, text="Genial", command=ventana_victoria.destroy, width=15, bg="#4cd137", fg="white", font=("Segoe UI", 10, "bold"), relief="flat", cursor="hand2").pack(pady=10)
70
71 # Revisa si las tres mascotas ya tienen el estado "True" (rescatadas). Si es así, lanza la victoria.
72 def verificar_victoria():
73     if estado_juego["Lucky"]["rescatado"] and estado_juego["Chester"]["rescatado"] and estado_juego["Nala"]["rescatado"]:
74         mostrar_pantalla_victoria()
75         actualizar_interfaz()
76
77 # Cambia el color y el texto de los botones del menú cuando rescatas a una mascota.
78 def actualizar_interfaz():
79     color_lucky = "#7f8fa6" if estado_juego["Lucky"]["rescatado"] else "#00a08f"
80     color_chester = "#7f8fa6" if estado_juego["Chester"]["rescatado"] else "#4cd137"
81     color_nala = "#7f8fa6" if estado_juego["Nala"]["rescatado"] else "#e6b12c"
82
83     btn_lucky.config(text="✓ Lucky (RESCATADO)" if estado_juego["Lucky"]["rescatado"] else "📍 Zona 1: El Río (Lucky)", bg=color_lucky)
84     btn_chester.config(text="✓ Chester (RESCATADO)" if estado_juego["Chester"]["rescatado"] else "📍 Zona 2: Mariposario (Chester)", bg=color_chester)
85     btn_nala.config(text="✓ Nala (RESCATADO)" if estado_juego["Nala"]["rescatado"] else "📍 Zona 3: La Cabaña (Nala)", bg=color_nala)
86
87 # -----
88 # NIVELES DEL JUEGO (ZONAS Y COMPUERTAS)
89 # -----
90
91 # ZONA 1: Compuerta AND. Las dos condiciones deben ser ciertas para ganar.
92 def zona_rio_lucky():
93     # Si ya lo rescatamos, no nos deja volver a jugar este nivel.
94     if estado_juego["Lucky"]["rescatado"]:
95         messagebox.showinfo("Zona 1", "Tu perrito Lucky ya está a salvo y mueve su colita a tu lado.", parent=ventana)

```

```

projectFinalRpy X
Version Final > projectFinalRpy.py > estado_juego

92 def zona_rio_lucky():
93     if estado_juego["Lucky"]["rescatado"]:
94         messagebox.showinfo("Zona 1", "Tu perrito Lucky ya está a salvo y mueve su colita a tu lado.", parent=ventana)
95         return
96
97     Intro = ("Narrador: Lucky ha quedado atrapado al otro lado del río.\n\n"
98             "LECCIÓN LÓGICA: Conoce a la compuerta 'AND'\n\n"
99             "La regla es estricta. Exige que TODAS las condiciones sean verdaderas al mismo tiempo.\n\n"
100            "Para que Lucky cruce, necesitas tener comida (para atraerlo) Y una correa (para asegurarlo), si te falta alguna de las dos, no cruzará.")
101    messagebox.showinfo("Zona 1: Salva a Lucky", Intro, parent=ventana)
102
103    # Pedimos las respuestas pasando las instrucciones por si el jugador se equivoca.
104    resp_comida = pedir_respuesta_si_no("Compuerta AND", "¿Tienes comida en tu mochila? (sí/no):", Intro)
105    if resp_comida is None: return
106
107    resp_correa = pedir_respuesta_si_no("Compuerta AND", "¿Tienes una correa en tu mochila? (sí/no):", Intro)
108    if resp_correa is None: return
109
110    # Evaluamos con AND: si tiene comida Y tiene correa
111    if resp_comida == "si" and resp_correa == "si":
112        mensaje_exito = "¡VERDADERO + VERDADERO = VERDADERO!\n\n¡Excelente deducción! Al cumplir estrictamente ambas condiciones, tu perrito Lucky cruzó el río sin miedo y ya está a salvo"
113        messagebox.showinfo("Éxito lógico!", mensaje_exito, parent=ventana)
114        estado_juego["Lucky"]["rescatado"] = True
115    else:
116        val1 = "VERDADERO" if resp_comida == "si" else "FALSO"
117        val2 = "VERDADERO" if resp_correa == "si" else "FALSO"
118        mensaje_fallo = f"({val1}) + ({val2}) = FALSO!\n\nLa regla falló. Necesitas cumplir AMBAS condiciones a la vez. Intentálo de nuevo."
119        messagebox.showwarning("Fallo lógico", mensaje_fallo, parent=ventana)
120
121    verificar_victoria()
122
123 # ZONA 2: Compuerta OR. Basta con que una condición sea cierta para ganar.
124 def zona_mariposario_chester():
125     if estado_juego["Chester"]["rescatado"]:
126         messagebox.showinfo("Zona 2", "Tu perrito Chester ya está a salvo contigo pidiendo mimos.", parent=ventana)
127         return
128
129     Intro = ("Narrador: Chester está hipnotizado persiguiendo mariposas de colores ¡necesita salir de ese lugar!\n\n"
130            "LECCIÓN LÓGICA: Compuerta 'OR'\n\n"
131            "La regla es flexible. Funciona si AL MENOS UNA de las condiciones es verdadera o incluso si AMBAS lo son.\n\n"
132            "Chester irá contigo si le ofreces su premio favorito, si haces sonar su juguete ruidoso o los dos al mismo tiempo.")
133    messagebox.showinfo("Zona 2: Salva a Chester", Intro, parent=ventana)
134
135    resp_premio = pedir_respuesta_si_no("Compuerta OR", "¿Quieres ofrecerle su premio favorito? (sí/no):", Intro)
136    if resp_premio is None: return
137
138    resp_juguete = pedir_respuesta_si_no("Compuerta OR", "¿Quieres hacer sonar su juguete ruidoso? (sí/no):", Intro)
139    if resp_juguete is None: return

```

```
Terminal Help  ← →  proyecto-rescate-mindo  Update

◆ projectofinalPpy X
Version Final > ◆ projectofinalPpy > ☐ zona_rio_lucky

125 def zona_mariposario_chester():
140     if resp_juguete is None: return
141
142     val_premio = "VERDADERO" if resp_premio == "si" else "FALSO"
143     val_juguete = "VERDADERO" if resp_juguete == "si" else "FALSO"
144
145     # Evaluamos los tres casos posibles del OR para narrarlo mejor
146     if resp_premio == "si" and resp_juguete == "si":
147         mensaje_exito = f"¡{val_premio} + {val_juguete} = VERDADERO!\n\n¡Hiciste ambas cosas! Le ofreciste su premio y sonaste su juguete. Como la regla es flexible, permite que ambas ac
148         messagebox.showinfo("Exito logico!", mensaje_exito, parent=ventana)
149         estado_juego["Chester"]["rescatado"] = True
150     elif resp_premio == "si" or resp_juguete == "si":
151         mensaje_exito = f"¡{val_premio} + {val_juguete} = VERDADERO!\n\n¡Muy bien! Como la regla es flexible, bastó con que una sola acción funcionara. Tu perrito Chester dejó de perseg
152         messagebox.showinfo("Exito logico!", mensaje_exito, parent=ventana)
153         estado_juego["Chester"]["rescatado"] = True
154     else:
155         mensaje_fallo = f"¡{val_premio} + {val_juguete} = FALSO!\n\n¡No hiciste nada para llamar su atención, sigue persiguiendo a las mariposas."
156         messagebox.showwarning("Fallo logico", mensaje_fallo, parent=ventana)
157
158     verificar_victoria()
159
160 # ZONA 3: Compuerta XOR y Bucle FOR. Solo una opción debe ser cierta y hay 3 intentos.
161 def zona_cabana_nala():
162     if estado_juego["Nala"]["rescatado"]:
163         messagebox.showinfo("Zona 3", "Tu gatita Nala ya esta a salvo y ronroneando.", parent=ventana)
164         return
165
166     intro = ("Narrador: Nala está atrapada en una cabaña ¡necesitas abrir la puerta!\n\n"
167             "LECCIÓN LÓGICA: Compuerta 'XOR'\n\n"
168             "La regla exige exclusividad estricta. El resultado solo es VERDADERO si las entradas son diferentes.\n\n"
169             "Para abrir la puerta, debes presionar EXACTAMENTE UN botón (uno 'si' y el otro 'no').\n\n"
170             "• Si presionas ambos botones ('si' y 'si'), el sistema falla.\n\n"
171             "• Si no presionas ninguno de los botones ('no' y 'no'), el sistema también falla.\n\n"
172             "¡Tienes solo 3 intentos!")
173     messagebox.showinfo("Zona 3: Salva a Nala", intro, parent=ventana)
174
175     # Usamos un bucle for para darle 3 intentos al jugador
176     for intento in range(1, 4):
177         resp_rojo = pedir_respuesta_si_no(f"Intento {intento}/3", "¿Presionas el botón ROJO? (si/no):", intro)
178         if resp_rojo is None: return
179
180         resp_azul = pedir_respuesta_si_no(f"Intento {intento}/3", "¿Presionas el botón AZUL? (si/no):", intro)
181         if resp_azul is None: return
182
183         # Convertimos el texto a booleanos puros de Python
184         boton_rojo = (resp_rojo == "si")
185         boton_azul = (resp_azul == "si")
186
```

```
Terminal Help  ← →  proyecto-rescate-mindo  Update

◆ projectofinalPpy X
Version Final > ◆ projectofinalPpy > ☐ zona_cabana_nala

161 def zona_cabana_nala():
183     # Convertimos el texto a booleanos puros de Python
184     boton_rojo = (resp_rojo == "si")
185     boton_azul = (resp_azul == "si")
186
187     val_rojo = "VERDADERO" if boton_rojo else "FALSO"
188     val_azul = "VERDADERO" if boton_azul else "FALSO"
189
190     # El símbolo ^ evalúa la lógica XOR en Python
191     if boton_rojo ^ boton_azul:
192         mensaje_exito = f"¡{val_rojo} + {val_azul} = VERDADERO!\n\n¡Lógica perfecta! Al presionar solo un botón, la puerta se abrió. Tu gatita Nala saltó libre de la cabaña y está a
193         messagebox.showinfo("Exito logico!", mensaje_exito, parent=ventana)
194         estado_juego["Nala"]["rescatado"] = True
195         break # Detenemos el bucle porque ya ganó
196     else:
197         # Si falló, revisamos si le quedan intentos
198         if intento < 3:
199             mensaje_fallo = f"¡{val_rojo} + {val_azul} = FALSO!\n\n¡Error de exclusividad! Recuerda: las entradas deben ser diferentes. La puerta sigue cerrada."
200             messagebox.showwarning("Fallo logico", mensaje_fallo, parent=ventana)
201         else:
202             messagebox.showerror("Bloqueo del Sistema", "Has fallado 3 veces. El sistema de la cabaña se ha bloqueado temporalmente, vuelvelo a intentar.", parent=ventana)
203
204     verificar_victoria()
205
206 # -----
207 # DISEÑO DE LA INTERFAZ GRÁFICA
208 # -----
209 ventana = tk.Tk()
210 ventana.title("Mindologic - Aprendiendo Lógica de Programación")
211 ventana.geometry("500x560")
212 ventana.configure(bg="#2f3640")
213
214 # Estilos de texto
215 fuente_titulo = ("Segoe UI", 16, "bold")
216 fuente_sub = ("Segoe UI", 10)
217 fuente_nota = ("Segoe UI", 9, "italic")
218 fuente_btn = ("Segoe UI", 12, "bold")
219
220 # Títulos principales y notas (Aquí sí van los emojis decorativos)
221 tk.Label(ventana, text="🧠 Mindologic: Misión Rescate en Mindo", font=fuente_titulo, bg="#2f3640", fg="#fbc531").pack(pady=(25, 5))
222 tk.Label(ventana, text="📖 Aprende cómo piensan las computadoras rescatando mascotas y utilizando las compuertas lógicas (AND, OR y XOR).", font=fuente_sub, bg="#2f3640", fg="#dcdde1").pack(pady=(10, 5))
223 tk.Label(ventana, text="📌 Nota: El sistema solo aceptará las respuestas solicitadas (ej. 'si' o 'no').", font=fuente_nota, bg="#2f3640", fg="#fff7675").pack(pady=(0, 20))
224
225 # Agrupamos los botones de los niveles en este contenedor
226 frame_zonas = tk.Frame(ventana, bg="#2f3640")
227 frame_zonas.pack(pady=10)
228
229 # Creamos los botones y los enlazamos a sus respectivas funciones
230
231 # Creamos los botones y los enlazamos a sus respectivas funciones
232 btn_lucky = tk.Button(frame_zonas, text="🍀 Zona 1: El Río (Lucky)", command=zona_rio_lucky, width=30, bg="#00a8ff", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=8)
233 btn_lucky.pack(pady=10)
234 btn_chester = tk.Button(frame_zonas, text="🐕 Zona 2: Mariposario (Chester)", command=zona_mariposario_chester, width=30, bg="#4cd137", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=10)
235 btn_chester.pack(pady=10)
236 btn_nala = tk.Button(frame_zonas, text="🐈 Zona 3: La Cabaña (Nala)", command=zona_cabana_nala, width=30, bg="#e1b12c", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=10)
237 btn_nala.pack(pady=10)
238
239 # Una línea gris decorativa para separar los niveles del botón de reinicio
240 tk.Frame(ventana, height=1, bg="#7f8fa6", width=400).pack(pady=15)
241
242 # Botón para reiniciar
243 btn_reinicio = tk.Button(ventana, text="🔄 Reiniciar Nivel", command=reiniciar_juego, width=20, bg="#e84118", fg="white", font=("Segoe UI", 11, "bold"), relief="flat", cursor="hand2", pady=10)
244 btn_reinicio.pack()
245
246 # Mantiene la ventana abierta esperando a que el jugador haga clic
247 ventana.mainloop()
```

```
Terminal Help  ← →  proyecto-rescate-mindo  Update

◆ projectofinalPpy X
Version Final > ◆ projectofinalPpy > ☐ zona_cabana_nala

229 # Creamos los botones y los enlazamos a sus respectivas funciones
230 btn_lucky = tk.Button(frame_zonas, text="🍀 Zona 1: El Río (Lucky)", command=zona_rio_lucky, width=30, bg="#00a8ff", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=8)
231 btn_lucky.pack(pady=10)
232
233 btn_chester = tk.Button(frame_zonas, text="🐕 Zona 2: Mariposario (Chester)", command=zona_mariposario_chester, width=30, bg="#4cd137", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=10)
234 btn_chester.pack(pady=10)
235
236 btn_nala = tk.Button(frame_zonas, text="🐈 Zona 3: La Cabaña (Nala)", command=zona_cabana_nala, width=30, bg="#e1b12c", fg="white", font=fuente_btn, relief="flat", cursor="hand2", pady=10)
237 btn_nala.pack(pady=10)
238
239 # Una línea gris decorativa para separar los niveles del botón de reinicio
240 tk.Frame(ventana, height=1, bg="#7f8fa6", width=400).pack(pady=15)
241
242 # Botón para reiniciar
243 btn_reinicio = tk.Button(ventana, text="🔄 Reiniciar Nivel", command=reiniciar_juego, width=20, bg="#e84118", fg="white", font=("Segoe UI", 11, "bold"), relief="flat", cursor="hand2", pady=10)
244 btn_reinicio.pack()
245
246 # Mantiene la ventana abierta esperando a que el jugador haga clic
247 ventana.mainloop()
```

### **iii. Estructura Lógica del Programa**

#### **1. Inicialización**

- Se cargan las librerías
- Se verifica que se tenga la librería de Pillow necesaria para cargar la imagen final del juego, en caso de que no se tenga, se descarga automáticamente
- Se crea el estado inicial del juego

#### **2. Definición de Funciones**

- Se declaran todas las funciones que se van a encargar de controlar todo el comportamiento del sistema

#### **3. Construcción de la Interfaz**

- Se crea la ventana principal, los botones y todos los textos

#### **4. Ejecución del Juego**

- El Software espera que el usuario interactúe con los botones del sistema

#### **5. Procesamiento de Cada Nivel**

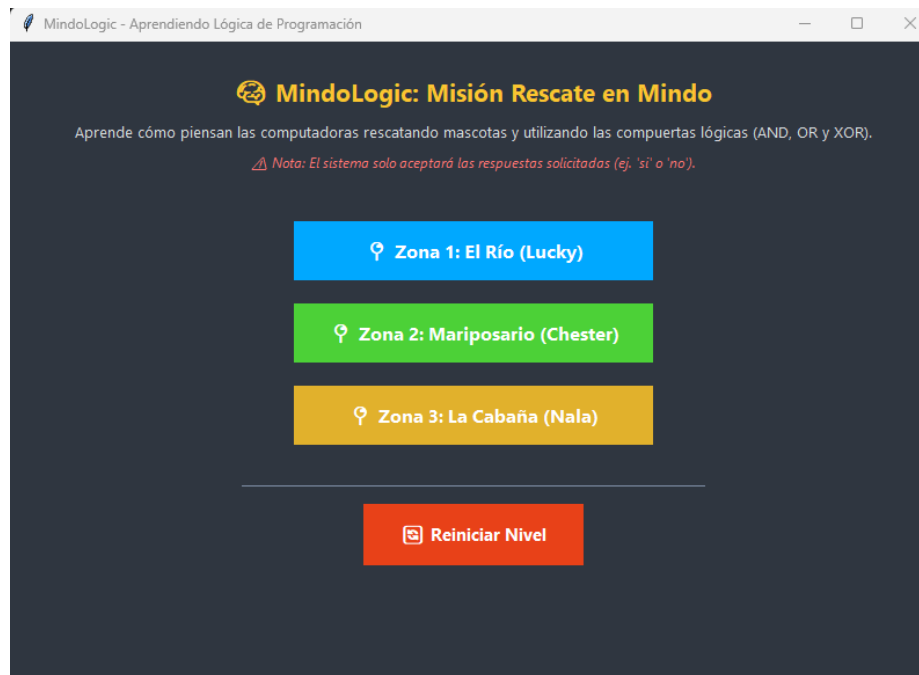
- Según la zona que el usuario escoja se solicitarán diferentes respuestas, se evalúan las diferentes condiciones lógicas (AND, OR o XOR), se actualiza el estado del juego y se comprueba si ya se ha completado el rescate de las tres mascotas

#### **6. Finalización**

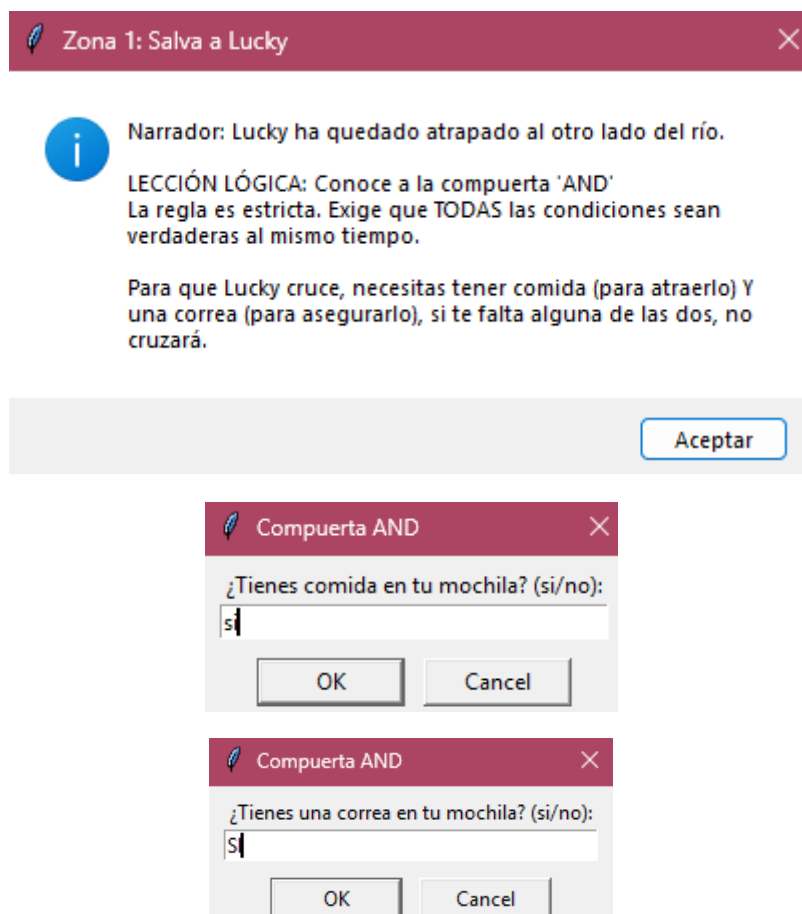
- Cuando las tres mascotas ya hayan sido rescatadas el programa mostrará una pantalla de victoria, caso contrario, el programa sigue esperando alguna acción del usuario

## VIII. EJECUCIONES

### i. Ejecución sin Restricciones



- Ejecución de la Zona 1 (Lucky)



✎ Exito logico!

**i** ¡VERDADERO + VERDADERO = VERDADERO!

¡Excelente deducción! Al cumplir estrictamente ambas condiciones, tu perrito Lucky cruzó el río sin miedo y ya está a salvo en tus brazos.

¡Lograste cumplir con la lógica de la compuerta AND!

Aceptar

MindoLogic - Aprendiendo Lógica de Programación

### MindoLogic: Misión Rescate en Mindo

Aprende cómo piensan las computadoras rescatando mascotas y utilizando las compuertas lógicas (AND, OR y XOR).

*⚠ Nota: El sistema solo aceptará las respuestas solicitadas (ej. 'si' o 'no').*

✓ Lucky (RESCATADO)

📍 Zona 2: Mariposario (Chester)

📍 Zona 3: La Cabaña (Nala)

 Reiniciar Nivel

- Ejecución de la Zona 3 (Nala)

✎ Zona 3: Salva a Nala

**i** Narrador: Nala está atrapada en una cabaña ¡necitas abrir la puerta!.

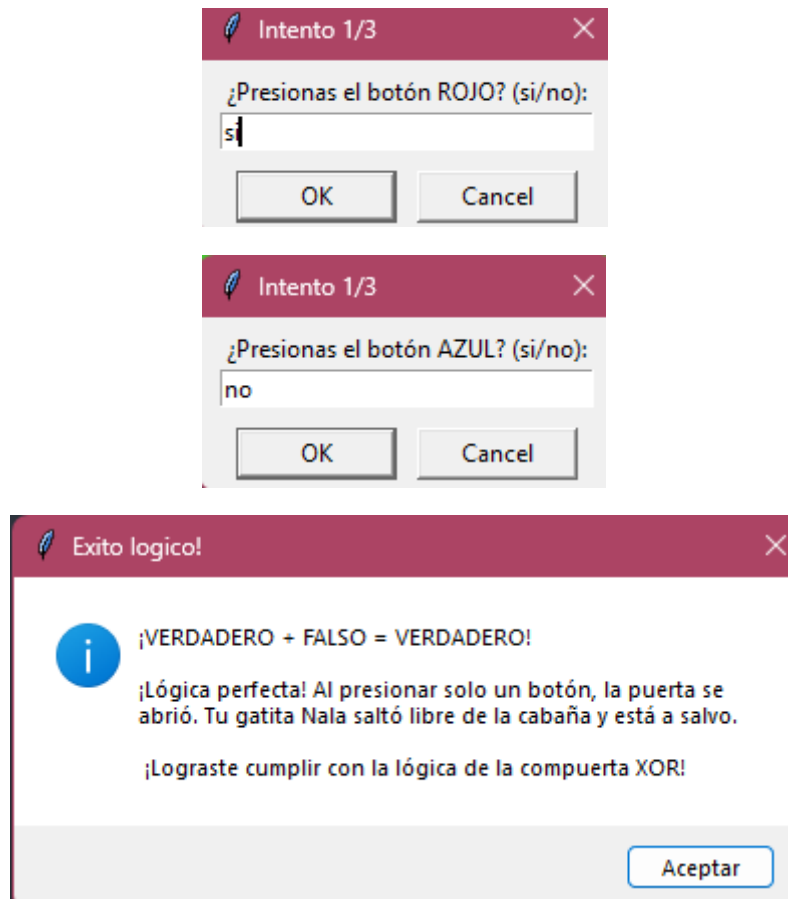
**LECCIÓN LÓGICA: Compuerta 'XOR'**  
La regla exige exclusividad estricta. El resultado solo es VERDADERO si las entradas son diferentes.

Para abrir la puerta, debes presionar EXACTAMENTE UN botón (uno 'si' y el otro 'no').

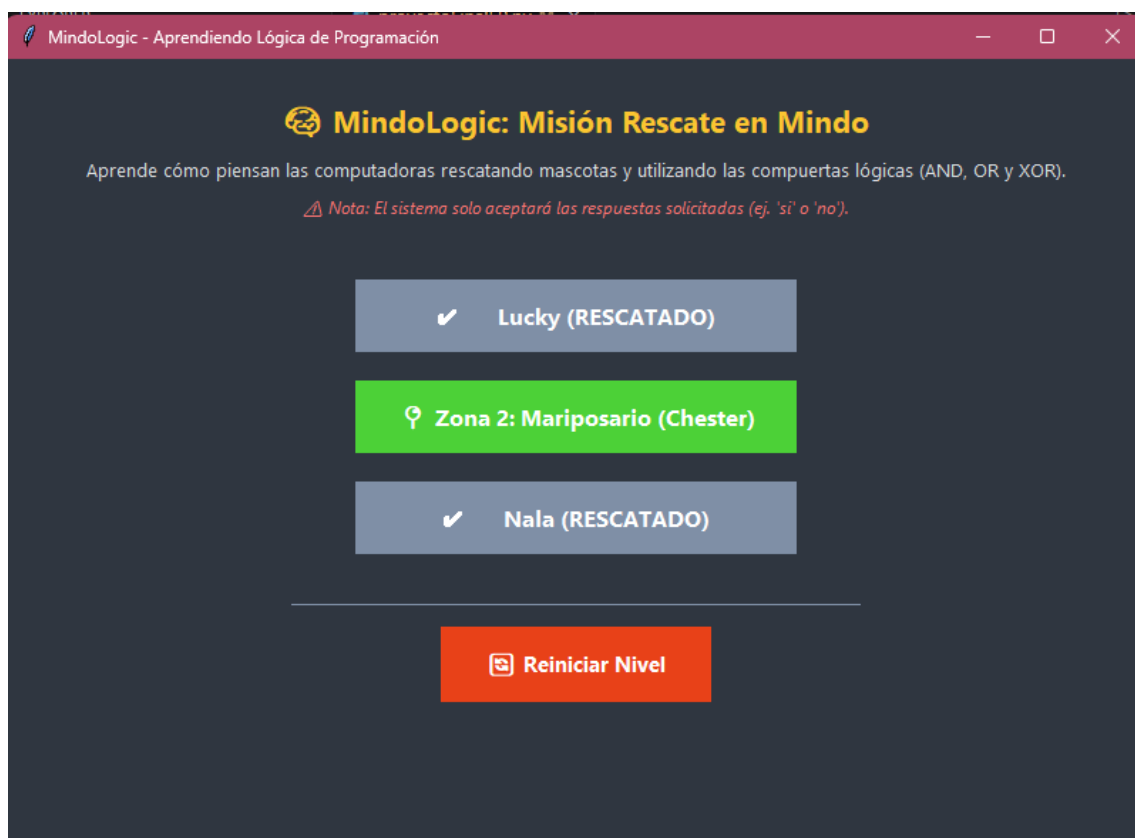
- Si presionas ambos botones ('si' y 'si'), el sistema falla.
- Si no presionas ninguno de los botones ('no' y 'no'), el sistema también falla.

¡Tienes solo 3 intentos!

Aceptar



- Primera solución de la Zona 2 (Chester)



 Zona 2: Salva a Chester 



Narrador: Chester está hipnotizado persiguiendo mariposas de colores ¡necesita salir de ese lugar!.

LECCIÓN LÓGICA: Compuerta 'OR'

La regla es flexible. Funciona si AL MENOS UNA de las condiciones es verdadera o incluso si AMBAS lo son.

Chester irá contigo si le ofreces su premio favorito, si haces sonar su juguete ruidoso o los dos al mismo tiempo.

Aceptar

 Compuerta OR 

¿Quieres ofrecerle su premio favorito? (si/no):

Si

OK

Cancel

 Compuerta OR 

¿Quieres hacer sonar su juguete ruidoso? (si/no):

no

OK

Cancel

 Exito logico! 



¡VERDADERO + FALSO = VERDADERO!

¡Muy bien! Como la regla es flexible, bastó con que una sola acción funcionara. Tu perrito Chester dejó de perseguir mariposas y corrió feliz hacia ti.

¡Lograste cumplir con la lógica de la compuerta OR!

Aceptar

- Segunda solución de la Zona 2 (Chester)

 Zona 2: Salva a Chester 



Narrador: Chester está hipnotizado persiguiendo mariposas de colores ¡necesita salir de ese lugar!.

LECCIÓN LÓGICA: Compuerta 'OR'

La regla es flexible. Funciona si AL MENOS UNA de las condiciones es verdadera o incluso si AMBAS lo son.

Chester irá contigo si le ofreces su premio favorito, si haces sonar su juguete ruidoso o los dos al mismo tiempo.

Aceptar

Compuerta OR

¿Quieres ofrecerle su premio favorito? (si/no):

si

OK Cancel

Compuerta OR

¿Quieres hacer sonar su juguete ruidoso? (si/no):

si

OK Cancel

Exito logico!

¡VERDADERO + VERDADERO = VERDADERO!

¡Hiciste ambas cosas! Le ofreciste su premio y sonaste su juguete. Como la regla es flexible, permite que ambas acciones sean verdaderas. ¡Chester corrió rapidísimo hacia ti!

¡Lograste cumplir con la lógica de la compuerta OR!

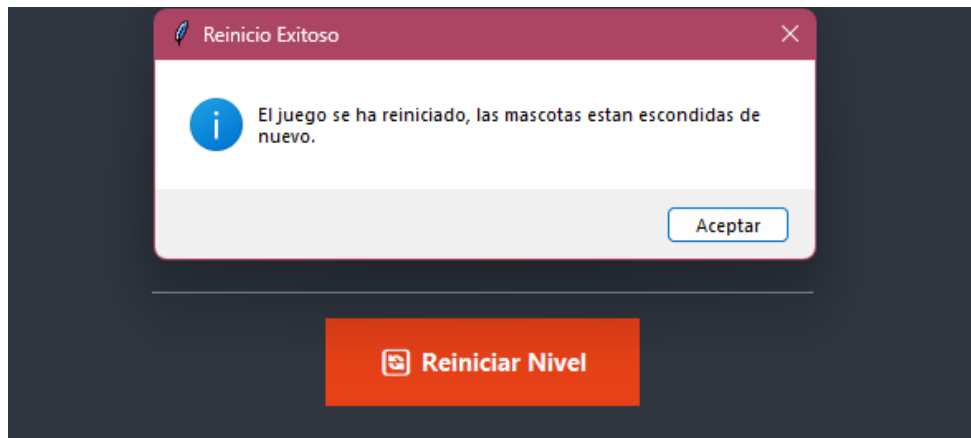
Aceptar



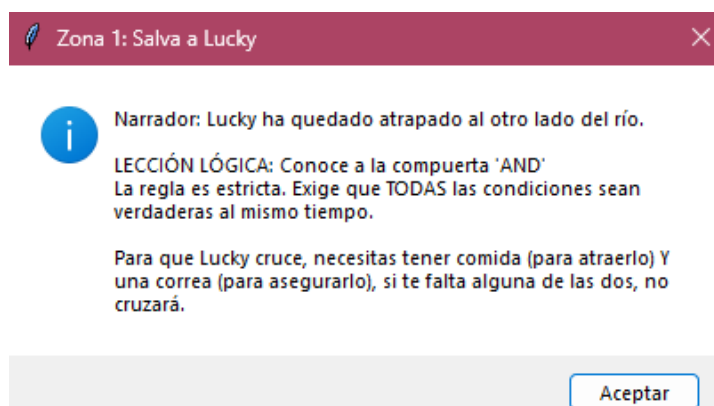


## ii. Ejecución con Restricciones

- Reiniciamos el juego para mostrar ahora algunos mensajes que nos ayudan a controlar los fallos que puede originar el usuario dentro del sistema



- Ejecución de la Zona 1 (Lucky)



- Ingreso de un carácter no definido cómo respuesta aceptada

Compuerta AND

¿Tienes comida en tu mochila? (si/no):

M

OK Cancel

Entrada invalida

Por favor, responde estrictamente con 'si' o 'no'.

Aceptar

- Incumplimiento con la compuerta lógica “AND”

Compuerta AND

¿Tienes comida en tu mochila? (si/no):

si

OK Cancel

Compuerta AND

¿Tienes una correa en tu mochila? (si/no):

nd

OK Cancel

Fallo logico

¡VERDADERO + FALSO = FALSO!  
La regla falló. Necesitas cumplir AMBAS condiciones a la vez.  
Inténtalo de nuevo.

Aceptar

- Este aviso es en caso de que queramos volver a jugar una zona que ya anteriormente fue resuelta

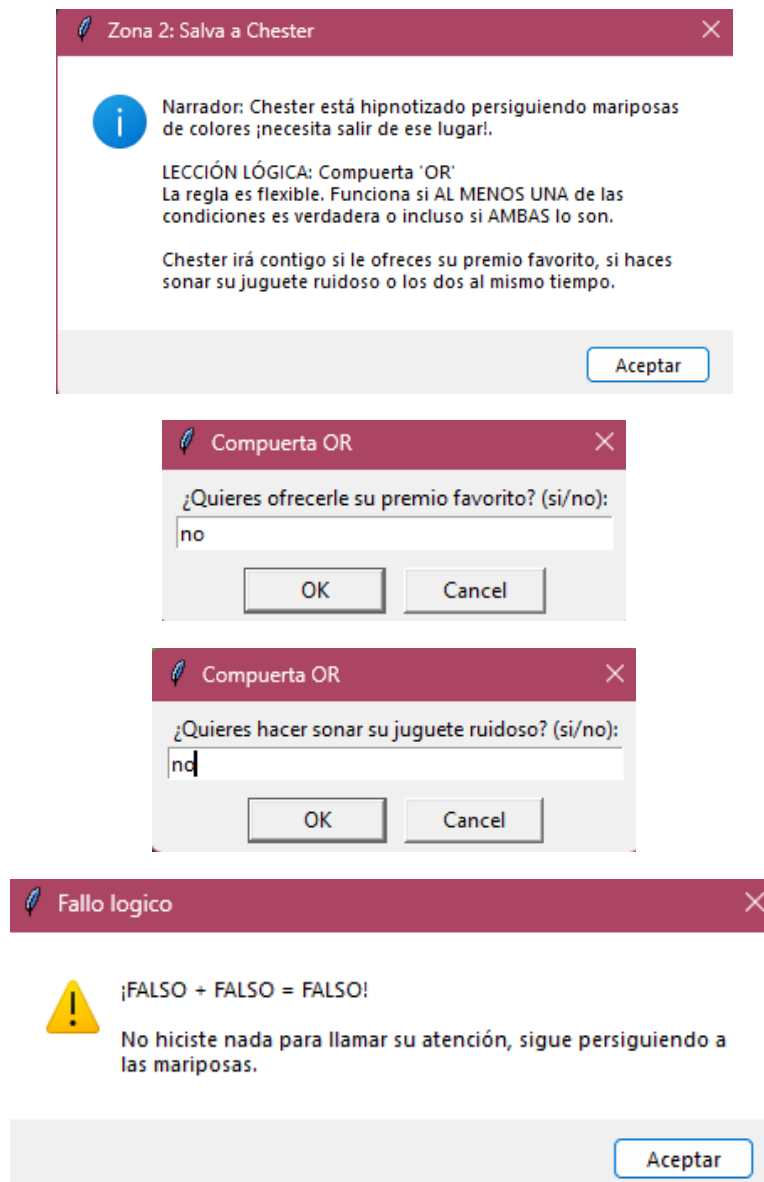
✓ Lucky (RESCATADO)

Zona 1

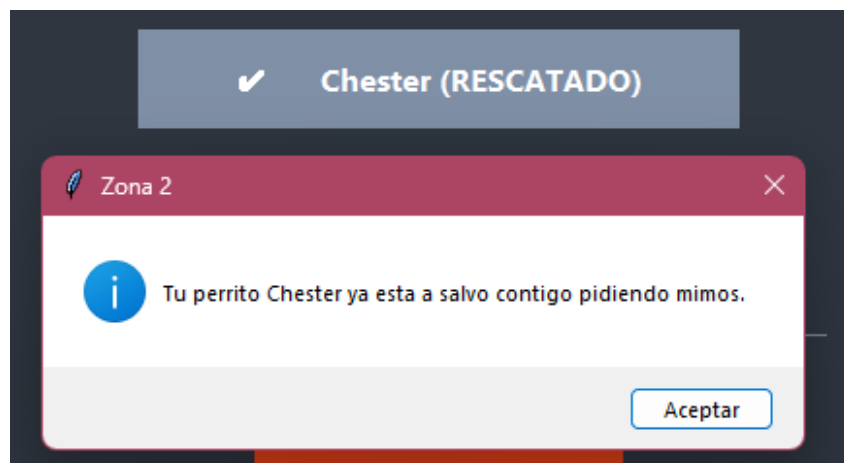
Tu perrito Lucky ya esta a salvo y mueve su colita a tu lado.

Aceptar

- Ejecución de la Zona 2 (Chester) e incumplimiento de la compuerta lógica “OR”



- Intento de ingreso a una zona ya resuelta



- Ejecución de la Zona 3 (Nala) e incumplimiento de la compuerta lógica “XOR”

Zona 3: Salva a Nala

i

Narrador: Nala está atrapada en una cabaña ¡necitas abrir la puerta!.

LECCIÓN LÓGICA: Compuerta 'XOR'

La regla exige exclusividad estricta. El resultado solo es VERDADERO si las entradas son diferentes.

Para abrir la puerta, debes presionar EXACTAMENTE UN botón (uno 'si' y el otro 'no').

- Si presionas ambos botones ('si' y 'si'), el sistema falla.
- Si no presionas ninguno de los botones ('no' y 'no'), el sistema también falla.

¡Tienes solo 3 intentos!

Aceptar

Intento 1/3

¿Presionas el botón ROJO? (si/no):

no

OKCancel

Intento 1/3

¿Presionas el botón AZUL? (si/no):

no

OKCancel

Fallo logico

!

¡FALSO + FALSO = FALSO!

¡Error de exclusividad! Recuerda: las entradas deben ser diferentes. La puerta sigue cerrada.

Aceptar

Intento 2/3

¿Presionas el botón ROJO? (si/no):

si

OK Cancel

Intento 2/3

¿Presionas el botón AZUL? (si/no):

si

OK Cancel

Fallo logico

¡VERDADERO + VERDADERO = FALSO!

¡Error de exclusividad! Recuerda: las entradas deben ser diferentes. La puerta sigue cerrada.

Aceptar

- Bloqueo temporal del sistema al haber fallado las 3 oportunidades de desbloquear la puerta.

Bloqueo del Sistema

Has fallado 3 veces. El sistema de la cabaña se ha bloqueado temporalmente, vuelvelo a intentar.

Aceptar

## **IX. ENLACE DEL REPOSITORIO**

- <https://github.com/NicoleSani04/proyecto-rescate-mindo>

## **X. ENLACE DEL VÍDEO**

- [https://youtu.be/MADQw8eb\\_k8](https://youtu.be/MADQw8eb_k8)

## **XI. CONCLUSIONES**

- El presente trabajo de investigación y desarrollo nos permite concluir que la base estructural de cualquier software recae en el correcto uso de la lógica computacional, trasladar reglas de negocio a expresiones booleanas compuestas, nos demuestra que es el fundamento esencial para que nuestra máquina pueda tomar diferentes decisiones de forma precisa.
- La evolución de un entorno enteramente de línea de comandos (CLI) hacia una interfaz gráfica (GUI) comprobó que, con la adopción correcta de las estructuras de datos y la encapsulación mediante funciones, no solo nos ayuda a optimizar el rendimiento y el uso de la memoria, sino que también nos facilita la escalabilidad futura del código a pesar de haber realizado un gran cambio dentro de la aplicación.
- Concluimos que el control de flujo estructurado, es decir el uso de condicionales e iteraciones, nos ayuda a tener un programa resiliente para gestionar la imprevisibilidad del factor humano, evitándonos así errores de ejecución mediante un diseño que siempre busca prevenir.

## **XII. RECOMENDACIONES**

- Al momento de hacer códigos tan grandes y con tantas aristas, se recomienda hacer diferentes pruebas dentro de la aplicación, inclusive incitar personas externas para ver su comportamiento con el programa sin haberles dado uno mismo ninguna instrucción, esto para ver si el programa es claro, intuitivo y verificar diferentes errores que se pueden presentar dentro del mismo.
- Se recomienda la realización del diseño de los diagramas de flujo y de arquitectura, esto antes de la escritura del código, ya que esta práctica demostró ser crucial para poder mapear la complejidad cognitiva de nuestro programa, antes de ser integrado a un entorno gráfico.

## **XIII. BIBLIOGRAFÍA**

- [ 1 ] Bucle while en Python. (s/f). El Libro De Python. Recuperado el 25 de junio de 2026, de <https://ellibrodepython.com/while-python>
- [ 2 ] Bucle for. (s/f). El Libro De Python. Recuperado el 25 de junio de 2026, de <https://ellibrodepython.com/for-python>
- [ 3 ] Source control in VS code. (s/f). Visualstudio.com. Recuperado el 25 de junio de 2026, de <https://code.visualstudio.com/docs/sourcecontrol/overview>